

One Reason to Change

CSCI-121 · Elements of Computer Programming II

Week 1, Day 2 · Thursday, August 27 · SC 333

What you leave with today

- A **usable test** for whether a class is doing too much
- You have read a real class diagram of **the system you will build**
- You have decomposed a problem and drawn it, splitting where the test says split
- **You have pushed a commit to GitHub** — and you know why commit alone is invisible to me

Laptops out for the last fifteen minutes. Everything before that is paper.

First, Tuesday

in your own words — from your exit tickets

The three parts of an object

	The question it answers
Identity	Which <i>one</i> is it?
State	What is true about it right now?
Behavior	What is it responsible for?

A thing with **no behavior of its own** is usually state on something else.

Three things I got wrong on Wednesday

The recitation notes are published. Three sections are marked **Correction** — what I said in the room was a useful first approximation and not precisely right.

I said

HTTP splits your data into packets and guarantees delivery

private is about which objects can see each other at runtime

The stack lives on the CPU

Actually

TCP does that. HTTP sits on top and inherits the guarantee

It is enforced by the **compiler**, and the unit is the **class**

Both stack and heap are **RAM**. The stack is just small, reused, and usually cached

Read them. I would rather you learn in week one that I correct myself than find out in week ten.

When should one object be two?

the question Tuesday left open



A class should have one reason to change.

Not one method.

Not one job you can describe in a sentence if you talk fast.

One reason you would ever come back and edit this file.

The test

- Say out loud **everything the class does**
- Count how many **different forces** would make you edit it
- More than one → it should probably be more than one class

You can run this test on a class you have never seen, in about ninety seconds, without compiling anything.

FleetManager — what does it do?

```
public void    add(Vehicle vehicle)
public Vehicle findByPlate(String plate)
public double  totalDailyRevenue()
public String  buildHtmlReport()
public void    emailReportToOwner(String addr)
public void    chargeAllElectricVehicles()
```

How many reasons to change?

At least three

The force	What it makes you edit
The fleet rules change	new vehicle type, new search — add, findByPlate
The report layout changes	buildHtmlReport — <i>why does a fleet manager know what HTML is?</i>
The mail system changes	emailReportToOwner — <i>why does a fleet manager know about email?</i>

This class **works**. Every method is correct. It is still badly designed — and you can see that without running it.

Split it – one reason each

FleetManager
<code>add()</code> <code>findByPlate()</code> <code>totalDailyRevenue()</code>

the fleet

FleetReport
<code>buildHtmlReport()</code>

how it looks

ReportMailer
<code>send(String, String)</code>

how it leaves

Working code and good design are different questions. A test suite answers the first. Only you answer the second.

You already did this on Wednesday — smaller

```
private double balance;

public void deposit(double amount) {
    if (amount <= 0) throw new IllegalArgumentException...;
    balance += amount;
}
```

private decides **what** may touch your state.

One reason to change decides **why** the class exists at all.

Same instinct, two sizes: you are deciding **what belongs inside a boundary**. Wednesday you drew it around a field. Today you are drawing it around a class.

Read a real diagram

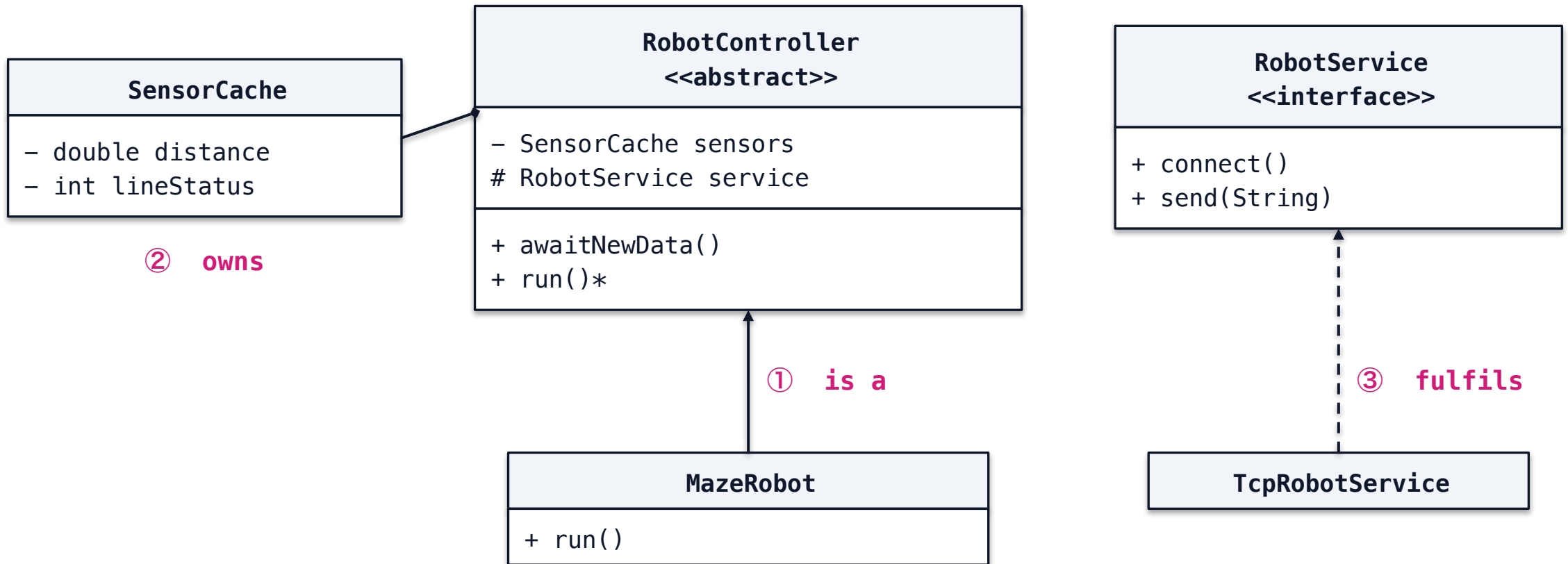
this is the robot you will build

Five shapes — that is all you need today

Shape	Means
Box with three compartments	a class — name, state, behavior
Solid arrow, hollow head	“is a” — the pointed-at one is the parent
Dotted arrow, hollow head	“fulfils a contract”
Filled diamond	“owns” — destroy the owner and this dies too
Hollow diamond	“has” — this outlives the owner

Every arrow in the diagram on the other screen is something you will understand by November. Today: just the shapes.

BRACE – three relationships, no more



① the parent connects to the robot — **you write the child**

② shut down the controller, the cache is gone

③ a promise kept over the network — someone could keep it differently

So — what happens here?

You are writing MazeRobot. It **extends** RobotController. You want the distance reading, so you write:

```
double d = sensors.distance;
```

Wednesday's rule. What happens?

It does not compile. `private` is enforced by the **compiler**, and the unit is the **class** — MazeRobot is a different class, subclass or not.

That is not the framework being unhelpful. Somebody decided you should go through `awaitNewData()` so you always read **one consistent snapshot** instead of four fields that changed underneath you mid-read.

Your turn

pairs · paper · laptops closed

The campus bike share

Bikes live at **docking stations**. A student unlocks a bike with their ID, rides it, and returns it to **any** station. Bikes need maintenance after a set number of rides. Staff need to know which stations are empty or full.

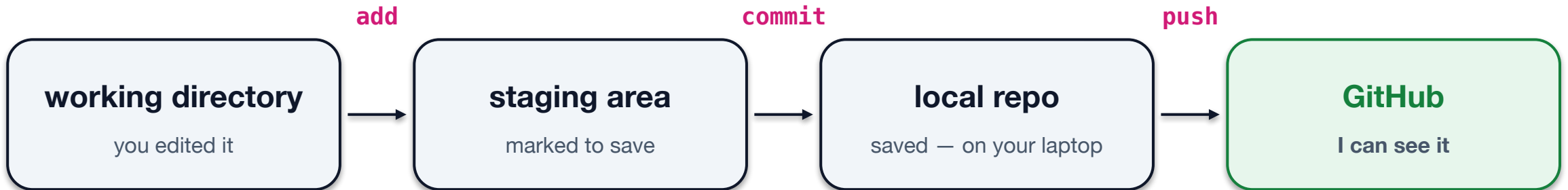
- List the objects
- For each: **identity, state, behavior**
- **Apply the test.** Is any one of them doing more than one job? Split it
- Draw the boxes. Connect two with an arrow, **labelled in plain English**

Read your arrow out loud as a sentence. If it is not a sentence, it is not a relationship yet.

Now hand it in

add · commit · push

Where your work can be



all three of these are on your laptop

Committed is not submitted. You did the work, you committed it, you never pushed — as far as the gradebook is concerned, you did nothing. This is the most common way to lose points in this class.

Four commands

`git status`

run this constantly – it tells you which box you are in

`git add design.md`

stage one file. `git add .` stages everything – careful

`git commit -m "Add bike share class diagram"`

saves it – on your laptop only

`git push`

now it exists for me

Commit messages: **present tense, say what changed.** Add bike share class diagram — not stuff, not asdf.

push will ask for a password

Your GitHub password will not work.

It has not worked since 2021. It wants a **Personal Access Token**.

```
GitHub → Settings → Developer settings  
→ Personal access tokens → Tokens (classic)  
→ Generate new token → check repo → Generate
```

copy it NOW – it is shown once, and never again

Paste the token where it asks for a **password**. If you get Authentication failed tonight, that is what it is — not your fork, not your clone. The token.

Do it now

- Type the diagram you just drew into `design.md`
- `git add design.md` → `git commit -m "..."` → `git push`
- **Refresh the GitHub page in your browser and see it there**

That last step is the point. The work is not done until you have looked at it **on the server**.

No laptop? Pair with someone who has one, and finish it tonight — §11 of the recitation notes.

Exit ticket — hand in on your way out

Find a class in the BRACE diagram with one clear responsibility.

Name it, and say what that responsibility is — **in one sentence.**

Bottom of your handout. Hand it to me as you leave.

Before you go

- **Assignment 00 was due before today.** Not pushed? Push it tonight — and if you are stuck, it is the token
- **Read the Recitation 00 notes** — especially the three **Correction** sections
- Section this week is **Recitation 0. Laptop + GitHub username required**
- In section you write the responsibility sentence for an object of your own, and draw it as a diagram you commit
- **The Robocode arena is still open** — optional, no hardware, runs all semester

Tuesday: Java, properly. Everything you know from Python — and the four places it lies to you.

The diagram specifies structure.

The test specifies behavior. The code satisfies both. — that is the whole course